

厦門大學

非 经 典 计 算
课 程 论 文

形式系统中的 λ 演算

λ Calculus in Formal Systems

姓 名：田佳铭
学 号：22920172204211
学 院：信息学院
专 业：智能科学与技术
年 级：2017 级

二〇二零年 5 月 17 日

摘要

作为一篇课程论文，本文的选题是“形式系统中的 λ 演算”，并且更加关注于从形式系统的角度来研究 λ 演算。作为一个可以说是最小的形式系统， λ 演算有着很多神奇的性质。本文首先介绍了形式主义思想的发展历程，主要关注于其哲学内涵。之后重点介绍 λ 演算的非形式定义和定义，并且通过 λ 演算和其他一些形式系统的对比来发现 λ 演算的优势。 λ 演算在现实中也是有应用的，例如函数式编程这种编程范式就是直接受到 λ 演算的影响。当然， λ 演算还有一些变体，例如 π 演算和 x 演算等，他们也在某种程度上影响到了我们的编程方式。之后，本论文还试图对形式系统、语言、计算和世界的概念和他们之间的关系给出自己的理解，也算是完成一项对自己这几年学到的知识的一个较为系统的总结吧。当然，在本文的最后，我也回答了同学们提出的一些问题。我坚信人工智能的未来是光明的，并会继续为人工智能的最终理想——理解人的智能而努力。

关键词： λ 演算；形式系统；语言；计算；人工智能

Abstract

As a course thesis, the topic of this article is " λ calculus in formal systems", and more attention is paid to studying λ calculus from the perspective of formal systems. As an arguably the smallest formal system, λ calculus has many magical properties. This article first introduces the development process of formalism, mainly focusing on its philosophical connotation. Afterwards, we focus on the informal definition and definition of λ calculus, and discover the advantages of λ calculus by comparing it with some other formal systems. The λ calculus is also applicable in reality. For example, the programming paradigm of functional programming is directly affected by the λ calculus. Of course, there are some variants of λ calculus, such as π calculus and χ calculus, etc., which also affect our programming to some extent. After that, this thesis also tried to give my own understanding of the concepts of formal systems, language, computing and the world and their relationship. It can be regarded as a more systematic summary of the knowledge I have learned in recent years. Of course, at the end of this article, I also answered some questions raised by the students. I firmly believe that the future of artificial intelligence is bright, and will continue to work hard for the ultimate ideal of artificial intelligence-understanding human intelligence.

Keywords: λ calculus; formal system; language; calculation; artificial intelligence

目录

1 引言.....	1
1.1 研究动机.....	1
1.2 研究目标.....	2
1.3 关于 λ 演算的简介.....	2
2 形式主义的历史.....	3
2.1 自柏拉图以来的理智主义传统.....	3
2.2 分析哲学、形式主义与数理逻辑.....	3
2.3 希尔伯特的 23 个问题.....	5
2.4 人工智能的产生与逻辑符号主义学派.....	6
2.5 早期人工智能研究领域的乐观主义者所坚信的四种假想.....	6
3 λ 演算的形式定义.....	9
3.1 语用层面（非形式定义）.....	9
3.1.1 匿名函数.....	9
3.1.2 柯里化.....	10
3.1.3 λ -记号表示法.....	10
3.1.4 λ -表达式.....	11
3.2 语法层面（形式定义）.....	11
3.2.1 符号字母表.....	11
3.2.2 λ 项.....	12
3.2.4 语法全等.....	12
3.2.5 自由变量与出现.....	13
3.2.6 替换.....	13
3.2.7 α -变换和 α -等价.....	14
3.2.8 β -规约.....	15
3.3 λ 演算与其他形式系统的比较.....	16
3.3.1 λ 演算与一阶谓词逻辑.....	17

3.3.2 λ 演算与元胞自动机.....	18
3.3.3 λ 演算与其他一些有关可计算函数的形式系统.....	19
4 λ 演算的应用初探.....	21
4.1 函数式编程.....	21
4.1.1 什么是函数式编程.....	21
4.1.2 函数式编程的优势与不足.....	22
4.2 λ 演算的变体.....	22
4.2.1 π 演算.....	22
4.2.2 χ 演算.....	23
5 形式系统、语言、世界与计算.....	25
5.1 维特根斯坦是谁.....	25
5.2 维特根斯坦前期理论、形式系统与世界.....	25
5.3 维特根斯坦后期理论、语言与世界.....	27
5.4 计算.....	28
5.5 形式系统、语言、世界与计算.....	30
6 对于报告问题的回应.....	33
7 写在后面的话.....	35
参考文献.....	36
附录 A: 希尔伯特问题与解决现状.....	37
附录 B: 关于命题逻辑的一些形式定义.....	39
附录 C: 关于形式系统 L 的一些形式定义.....	41
附录 D: 关于一阶谓词逻辑的一些形式定义.....	42
附录 E: 关于形式系统 K 的一些形式定义.....	45
附录 F: 关于 π 演算的一些形式定义.....	47
附录 G: χ 演算与 π 演算的比较.....	48

CONTENTS

1 Introduction.....	1
1.1 Research motivation.....	1
1.2 Research objectives.....	2
1.3 Introduction to λ Calculus.....	2
2 The history of formalism.....	3
2.1 The intellectualist tradition since Plato.....	3
2.2 Analytical philosophy, formalism and mathematical logic.....	3
2.3 Hilbert's 23 questions.....	5
2.4 The Generation of Artificial Intelligence and the School of Logical Symbolism.....	6
2.5 Four hypotheses strongly believed by optimists in the field of early artificial intelligence.....	6
3 Formal definition of λ calculus.....	9
3.1 Pragmatic level (informal definition).....	9
3.1.1 Anonymous function.....	9
3.1.2 Currying.....	10
3.1.3 λ -symbol notation.....	10
3.1.4 λ -expression.....	11
3.2 Grammatical level (formal definition).....	11
3.2.1 Symbol alphabet.....	11
3.2.2 λ term.....	12
3.2.4 Grammatical congruence.....	12
3.2.5 Free Variables and Appearance.....	13
3.2.6 Replace.....	13
3.2.7 α -transformation and α -equivalence.....	14
3.2.8 β -Statute.....	15
3.3 Comparison of λ calculus with other formal systems.....	16

3.3.1 Lambda calculus and first-order predicate logic.....	17
3.3.2 λ calculus and cellular automata.....	18
3.3.3 λ calculus and other formal systems related to computable functions.....	19
4 A Preliminary Study on the Application of λ Calculus.....	21
4.1 Functional programming.....	21
4.1.1 What is functional programming.....	21
4.1.2 The advantages and disadvantages of functional programming.	22
4.2 Variation of λ calculus.....	22
4.2.1 π calculus.....	22
4.2.2 $\times$ calculus.....	23
5 Formal system, language, world and computing.....	25
5.1 Who is Wittgenstein.....	25
5.2 Wittgenstein's Early Theory, Formal System and World.....	25
5.3 Wittgenstein's Later Theory, Language and World.....	27
5.4 Calculation.....	28
5.5 Formal system, language, world and computing.....	30
6 Response to some problems.....	33
7 Words written in the back.....	35
References.....	36
Appendix A: Hilbert Problem and Solution Status.....	37
Appendix B: Some formal definitions of propositional logic.....	39
Appendix C: Some formal definitions of formal system L.....	41
Appendix D: Some formal definitions of first-order predicate logic.....	42
Appendix E: Some formal definitions of formal system K.....	45
Appendix F: Some formal definitions of π calculus.....	47
Appendix G: Comparison between $\times$ calculus and π calculus	48

1 引言

1.1 研究动机

我很早以前就对 Lisp 感兴趣了，这种想法最早也许可以追溯到我接触到维特根斯坦的理论时，当时我感觉（前期）维特根斯坦的理论是浪漫的，就像洁白的冰原。慢慢的，从维特根斯坦开始，我接触到了分析哲学，并且通过数理逻辑，我将计算机与哲学思想结合了起来。在我看来，数理逻辑就像是连接两个世界的桥梁一样。

在人工智能导论 B 中，我第一次接触到了 lisp 语言，并用它来编写（从网上拷贝）了一些简单的程序，并且对这种语言产生了兴趣。上学期，我们在江敏老师的带领下学习了数理逻辑，这学期我也去修了哲学系的数理逻辑。虽然数理逻辑真的很难，但是我仍然从中学到了很多东西。这学期的语言分析技术中的编译原理部分也是研究形式语言的，并且，这门课也研究自然语言处理，这为我的研究提供了更多理论材料。

这学期的上的这门非经典计算课，也再次提到了 λ 演算，碰巧这学期我也在研究德雷福斯的人工智能批判理论，也离不开对逻辑符号主义的研究，于是就选择了 λ 演算作为本次论文与报告的主题。

我想通过这篇论文来整理一下我掌握的一些零散的理论 and 想法，形成一个较为较为完整的体系，但是因为时间和精力的限制，我也没有期望着能够做的特别完整，其中涉及到的很多观点也并不成熟，但是这种尝试我觉得是值得的。

我也经常问自己一个问题：为什么我总是喜欢去研究一些比较老的问题？确实，不管是早期维特根斯坦的理论（我觉得把它放在这里是没有问题的）、德雷福斯的人工智能批判理论还是 λ 演算本身，要说起来都是计算机科学或者人工智能发展历史上的古董了，但是我们仍然可以在现代的人工智能中窥探到他们的影子，他们并没有完全失去活力，我们仍然能发现它们的闪光点，体会前人的绝妙的想法。

本论文写作过程中我并不会用特别严谨的表述，也没有将一切论证都建立在论文证据的基础上，而是用这种比较口语化的表述，试图把事情说得浅显易懂。当然，作为一个课程论文，我也想将课堂上讲的一些知识和同学的一些报告整合

进来，来对本学期学到的知识有更好的把握。

1.2 研究目标

λ 演算的内容比较丰富，作为课程报告当然无法做出特别深入的研究，只能是初步踏入这个领域。

本次研究的重点也不在于函数式编程，而是紧扣 λ 演算本身，用形式系统的眼光去审视 λ 演算，比较 λ 演算和其他的一些形式系统的区别。

本次研究的另一个重要的组成部分就是从 λ 演算出发，放宽眼光，去重新理解形式系统及其背后的哲学思想。

1.3 关于 λ 演算的简介

λ 演算最早是由阿隆佐·丘奇（Alonzo Church）发明的^[1]，当时的目的也是为了实现数理逻辑的一些远大理想（这将在后文谈到），当然， λ 演算也并没有达到这个目的，很快被证明为是不相容的。

但是作为一个形式系统（它只包括一条变换规则和一条函数定义方式）， λ 演算本身就具有很多优良的性质。 λ 演算可以说是最简单、最小的一个形式系统，并且可以很方便的研究函数定义、函数应用和递归。而且， λ 演算是图灵完备的，和一般递归函数、图灵机等有着同等的能力。

由 λ 演算衍生出的函数式编程也是一种神奇的编程方法，最早应用 λ 演算一些编程语言，例如“神之语言”Lisp，有着很多连现在的编程语言都很难做到的神奇操作。函数式编程的优势渐渐地被更多人认识到了。

当然， λ 演算也有一些变体，例如 π 演算、 $\times$ 演算等，它们也是近年来一些并发程序的理论工具。^[1]

2 形式主义的历史

为了对 λ 演算有更加深刻的认识，我们需要先从哲学的角度出发去探究形式主义的历史。

2.1 自柏拉图以来的理智主义传统

形式主义有着很深的哲学渊源，甚至可以追溯到苏格拉底那里。

自从希腊人发明了逻辑和几何以来，就一直隐藏着某种想法——把一切推理都归结为某种计算，从而一劳永逸地解决所有的论证。这一想法最先被苏格拉底明确的提了出来。

在柏拉图那里，这种想法提升到认识论、甚至是本体论高度。我们可以从柏拉图的理智主义中看到这种思想的影子。柏拉图的理智主义认为我们可以把全部推理都规约为一些清晰的、离散的规则并且把世界规约为可以不需要解释便可以使用这些规则的离散的事实。^{[2]239}

虽然柏拉图的理智主义原本是为了概括道德上的确定性要求的，但是理智主义也确实影响到了其他哲学领域。自柏拉图以来，很多哲学家都发展了这一理论（当然嘛，整个哲学的历史甚至都可以说成是对柏拉图理论的不断解释）。

霍布斯第一次清晰地将思维的句法概念表达为计算^①。莱布尼兹也致力研究非二义的形式语言，从而将霍布斯的理论可以付诸实践。从这里开始，我们就可以发现在计算机出现之前，这些理论其实已经发展的比较成熟了。

2.2 分析哲学、形式主义与数理逻辑

十九世纪末，哲学界爆发了一场大革命，试图给当时已经即将陷入死寂的哲学以新生^②（虽然就结果而言，这场革命可能并没有突破传统逻辑的局限）。这

^① 当然，这不是对思维进行形式化的第一次努力，课堂上也介绍过十三世纪雷蒙·卢尔（Ramon Lull）的思维机（用逻辑的方法来探究神的思维），但是当时欧洲还处于神学统治一切的中世纪，就这点上来说，思维机和理智主义的哲学目标其实是有很大差别的。

^② 这其中很重要的原因是因为黑格尔提出了辩证法，并且宣称统一了一切哲学理论（这好似也宣

场革命产生了两个新的哲学派别，分析哲学就是其中一支。

简单的概括分析哲学的思想，即是说我们要将逻辑从心理学和认识论中分离出来，抛弃传统形而上学的观点，将一切归结于“分析”中。换言之，在分析哲学这里，分析是第一要务。

分析哲学的先驱是弗雷格（Frege）。分析哲学最开始的目标非常简单——我可以不知道你说的话是什么意思，但是我们可以计算一下它对不对。弗雷格提出的含义与指称理论，可以用下列的表格加以理解：

表 1 含义与指称的关系

表达式	含义	指称
专名	该专名的意义	对象
概念词	该概念词的意义	概念
句子	思想	真值

图表参考了厦门大学哲学系唐清涛老师在《现代西方哲学》课上关于分析哲学的课件（2019 年版）

直观上可以看出，含义与指称理论试图将现实中存在的事物与确定性的一些存在进行对应，其中很重要的一点就是，弗雷格认为句子的指称是真值。我们可以认为弗雷格在句子和真值之间做了一个映射，一个句子要么被映射成“真”，要么被映射成“假”^③。而句子的含义又是思想，即是说弗雷格也是要试图对思维进行分析。

在之后的发展过程中，分析哲学更加关注与分析本身。（以至于走得太远陷入了繁琐的分析过程之中）

形式主义最早由希尔伯特（Hilbert）在数学领域提出，他认为，数学是关于

布了哲学的灭亡）。在黑格尔这里，注重统一理论框架构建的哲学达到了巅峰，但是，人们逐渐发现，世界上依然有很多黑格尔的辩证法难以解决的问题，这种情况下爱，很多哲学家就开始探索哲学的新出路。

^③ 当然，在现在的一些非经典逻辑中，可以存在有不止真和假两种真值，例如“不确定”也是一种真值，但在经典逻辑中，还是主要以二值逻辑为主。

符号或有限的符号系统。希尔伯特想要构造的形式系统主要包括符号、公理和规则三部分。在这样的形式系统中，我们就可以凭借少数的公理和规则来一步步推导出大量的我们想要的结论。^{[3]12-13}

这种用简单的公理和规则来推演出无限的有用结论的思想并不是史无前例，就像在欧几里得的几何学公理体系，公理只有四条，却可以借助规则证明很多困难的几何问题。^[4]希尔伯特的主要想法是将这样的公理体系形式化地表达出来，这样，推理过程就可以自动的进行，而不需要借助于人去花费大量的经历去进行论证。

数理逻辑，是用数学方法研究逻辑或形式逻辑的学科。如果说形式主义是纯粹的数学中的产物，那么，数理逻辑就是将形式主义应用于逻辑中的产物，也就是说，我们可以认为数理逻辑是分析哲学和形式主义思想结合下的产物，在往上追溯上去，我们发现，数理逻辑并没有脱离自柏拉图以来的哲学传统，这种工具的出现也是为了研究逻辑，进而研究真理，从而实现用计算来一劳永逸地解决所有的论证的目的。

2.3 希尔伯特的 23 个问题

提到希尔伯特，就不得不提到希尔伯特问题。

在 1900 年巴黎国际数学家代表大会上，希尔伯特发表了题为《数学问题》的著名讲演。他根据过去特别是十九世纪数学研究的成果和发展趋势，提出了 23 个最重要的数学问题。这 23 个问题统称希尔伯特问题。这 23 个问题极大地推进了数学的发展，当然，也推进了数理逻辑的发展。

其中有些问题在形式逻辑中非常重要，例如第二个问题：算术公理系统的无矛盾性问题。简单来说，这个问题主要想做这么一件事情：证明真与可证是一致的。如果这个问题被证明成立，我们就可以用证明的方式来处理一切逻辑问题。

对于这个问题，哥德尔的不完备性定理给出了否定的答案。虽然有些学者认为我们高估了哥德尔的不完备性定理的作用，认为它只不过是说明了形式系统中的一些问题而已，但实际上，如果我们回看形式主义的思想脉络，我们就会发现，这几乎是给苏格拉底的理想来了一记深深的背刺。哥德尔不完备性定理告诉我

们：真与可证是两个概念，可证的一定是真的，真的不一定可证。形式主义的大厦从此崩坏了。

哥德尔的不完备性的影响当然不止于此，它也影响到了其他的领域，例如哲学、语言学、物理学。它甚至在隐隐告诉我们，我们可能永远找不到能一个解释世界的统一理论，关于这一点，我将在第五章继续阐述。

2.4 人工智能的产生与逻辑符号主义学派

计算机科学是从数学中分支出来的，同样以形式主义为纲领，从而最初的人工智能也是以形式主义为纲领的^④。^{[3]12}

人工智能领域，最早出现的学派就是逻辑符号主义学派。逻辑符号主义的哲学基础是数理逻辑、形式逻辑和分析哲学等哲学流派[7]，其思想内核就是形式主义。

逻辑符号主义认为，我们应该通过分析人类认知系统所具备的功能和机能，然后用计算机模拟这些功能。而模拟人的认知功能的方式即是试图将人的认知方式形式化，并且用形式系统的处理方式来模拟人的认知。

在下一节，我们会结合“早期人工智能研究领域的乐观主义者所坚信的四种假想”来具体分析这种思想的具体应用思路。

2.5 早期人工智能研究领域的乐观主义者所坚信的四种假想

在《计算机不能做什么——人工智能的极限》中，休伯特·德雷福斯（Hubert Dreyfus）提到了早期人工智能研究领域的乐观主义者所坚信的四种假想，即生物学假想、心理学假想、认识论假想和本体论假想^{[2]166}。

生物学假想是说，我们的大脑以离散的方式处理信息，大脑可以看成是一台精密的数字计算机。心理学假想是说，可以把大脑看成是一台按照形式规则加工信息单位的机器。认识论假想是说，一切知识都可以被形式化地表达出来。本体论假想是说，存在是一组在逻辑上相互独立的事实。

虽然德雷福斯总结出这四个假想的目的是为了对这四种假想进行批判，进而

^④ 关于人工智能的历史，这里就不多加叙述，大家都清楚了。

发展自己的人工智能批判理论，去尝试为人工智能找到新出路，进而帮助人工智能从根本上脱离困境^⑤，但是我们依然可以从这四个假想中对当时的人工智能研究状况有一定的认识。

包括《皇帝新脑》中也提到，强人工智能主义者对我们的大脑抱有很多“强假设”^[5]

也就是说，当时的人工智能研究者认为，虽然现实世界看起来是连续的，但是仍然存在着某种层次——不管是生物学上的还是心理学上的还是继续放大到本体论层面^⑥，在这个层次中，信息是被离散的进行处理的。这样，我们就可以用计算机来一一对应这种处理过程，进而模拟人脑^⑦乃至世界的运转，从而实现强人工智能的目标。

或者说，当时人工智能研究的主要思路是表征->形式推演->解释^⑧。一般来说，对结果的解释大致上可以看做是表征的逆过程，而形式推演又是计算机最擅长做的事情，所以在这个链条中，最重要的其实是表征。所谓表征，就是说我们要通过一定的手段，把真实世界中的场景转化成形式系统中的初始状态，甚至也包括形式推演规则的表征。我们对表征并不陌生，在用编程解决实际问题时，我们第一步做的就是表征的工作，我们写代码的过程就是在定义形式系统的初始状态和形式推演规则。

我们看到，命题逻辑和一阶谓词逻辑正是逻辑符号主义学派想要的东西，它们的目的就是把用以表示离散事实的命题即命题间的逻辑推论关系形式化地表达出来。所以，对于数理逻辑的研究和应用对于计算机来说是必不可少的。

到这里为止，我们就梳理出了一条非常重要的线索，这条线索从柏拉图的理智主义哲学思想开始，经过发展，进入不同的分支——其中比较重要的就是哲学领域的分析哲学、数学领域的形式主义和两者“杂交”出来的数理逻辑，这三者

^⑤ 德雷福斯的人工智能批评理论主要在上世纪 70 年代被提出，该理论一个重要的目标就是帮助人工智能走出低谷期。

^⑥ 我们可以看到，这四条假设呈现出了递进关系。

^⑦ 即逻辑符号主义学派当时研究人工智能的主要途径——认知模拟。

^⑧ 这也就是上一节提到的逻辑符号主义学派的主要研究思路。

促进了计算机、人工智能和逻辑符号主义学派的诞生，而且这一条传统最终被总结为四条假想^⑨指导下的人工智能发展的一条重要的支线^⑩。当然，现实中，这条支线的发展并没有那么简单，各种思想和工具之间都是相互影响、共同发展的。

当然，和计算机关系更密切的可以说是对可计算函数的研究，因为这和计算机的能力界限直接相关。课堂上也提到，有四种重要的和可计算函数有关的模型：一般递归函数、 λ 可计算函数（即 λ 演算）、图灵机、波斯特系统。接下来，我们就重点研究 λ 演算。

^⑨ 这四条假想的出现不止是依赖于哲学思想的指导和数学工具的出现，例如生物学假想

^⑩ 逻辑符号主义毕竟不是人工智能的全部，另外两个比较重要的学派是神经连接主义学派和控制论学派。

3 λ 演算的形式定义

虽然有很多关于 λ 演算的书中都介绍了关于 λ 演算的一些基本的定义,但是我觉得他们并不是严格符合我们学习数理逻辑中学习到的一般形式系统的定义方法,于是,我重新对 λ 演算的形式定义进行了梳理,当然,有些步骤我也参考了江敏老师的方法。

3.1 语用层面（非形式定义）

对于 λ 演算,我们需要先从语用层面了解 λ 演算到底是什么意思。

3.1.1 匿名函数

我们可以先考察任意一个函数^⑪,例如:

$$f(x, y) = x^2 + y \quad (1)$$

我们可以看到,这个函数是有函数名的。有函数名的好处就是我们可以在之后的计算中可以不会混淆不同的函数,方便我们的计算^⑫。

但是只是要分辨不同的函数的话,我们不一定需要给这个函数进行命名。例如,我们可以用下面这个映射来做到同样的事情:

$$(x, y) \mapsto x^2 + y \quad (2)$$

其中,左半部分是这个函数的参数,后半部分和(1)式等号右边的表达式是一样的(可以不用在意中间的那个符号,这只是一个过渡用的函数而已)。虽然这个函数看上去有些古怪,但是我们仍然可以像使用(1)式一样使用(2)式:

$$((x, y) \mapsto x^2 + y^2)(2, 3) = 2^2 + 3 = 7 \quad (3)$$

这看起来就像直接将(2)式当做函数名来使用一样。也就是说,(1)式和(2)式并没有本质上的区别,他们的能力是相等的。

我们称(2)式这样的函数称为匿名函数。

^⑪ 如无特殊说明,本文提到的函数都是可计算函数。

^⑫ 假设没有一个助记符让我们区分不同的函数,事情往往会变得比较麻烦。

3.1.2 柯里化

我们继续研究(2)式，我们发现它是一个二元函数，但是我们仍有办法改造一下它，进而得出以下的函数表达式：

$$x \mapsto (y \mapsto x^2 + y) \quad (4)$$

这个式子看起来更加奇怪了，但是我们仍然可以解释它，在这个函数中， x 被映射为 $y \mapsto x^2 + y$ ，后者仍然是一个函数，在这里，(4)式将 x 映射成了函数的函数，这样的函数被称为高阶函数。为了直观上说明(4)式和(2)式等价，我们可以用(3)式的赋值来进行代入，只不过，因为(4)式和 $y \mapsto x^2 + y$ 都是单参数函数，因此，这个过程需要分两步：

$$(x \mapsto (y \mapsto x^2 + y))(2) = y \mapsto 2^2 + y = y \mapsto 4 + y \quad (5)$$

$$(y \mapsto 4 + y)(3) = 4 + 3 = 7 \quad (6)$$

或者我们可以写成：

$$((x \mapsto (y \mapsto x^2 + y))(2))(3) = (y \mapsto 4 + y)(3) = 7 \quad (7)$$

可以看出，(4)式和(2)式是等价的，所以(4)式和(1)式也是等价的。

我们可以用这种方法，将任意一个函数转换为这种形式的单参数的高阶函数。结合匿名函数的思想，我们当然也可以将任意一个函数转换为这种形式的单参数的高阶匿名函数。

3.1.3 λ -记号表示法

但是这样的表述有时候会出现一些小问题，当然并不是说通过柯里化处理的公式和之前的公式不等价，而是因为按照(4)式的形式表达和调用函数可能会因为括号过多而看混。

丘奇于1941年提出 λ -记号表示法，它对无法使用或不打算使用助忆符命名的任意函数，给出标准的名称，使名称有一定意义。

使用 λ -记号就可以方便我们识别参变量。并且还要指出， λ -记号并不只能用于高阶函数。例如，将(2)式用 λ -记号表示为：

$$\lambda(x, y).x^2 + y \quad (8)$$

这样，我们就指明了这个函数中的变量为 x 和 y 对于(4)式，我们也可以使用 λ -记号表示：

$$\lambda x.(\lambda y.(x^2 + y)) \quad (9)$$

当然，对这个公式，在不引起歧义的情况下，我们可以将其改写成更加简洁的形式：

$$\lambda xy.(x^2 + y) \quad (10)$$

虽然看起来，(9)式和(7)式很像，但两者表达的意思是完全不同的，(10)本质上还是高阶函数，只不过是將一些符号省略掉了。

我们还可以用纯逻辑来表示(9)式：

$$\lambda xy.+((\times x)x)y \quad (11)$$

经过这样一系列的变换我们就把(1)式等价的表达为了(11)式，这种用 λ -记号表示法表示的高阶匿名函数就是我们要找的东西了。

3.1.4 λ -表达式

使用 λ -记号表示的函数就可以称为 λ -表达式。对于 λ -表达式，我们还要再深入认识一下它的结构，以方便后面我们对 λ 演算的理解。我们以(9)式为例。

在(9)式中， x, y 是约束变项。 $x^2 + y$ 是命名体。 λ 可以叫做函数抽象符，于是， $\lambda x.(\lambda y.(x^2 + y))$ 称为从 $x^2 + y$ 中(经二次)抽象所得的函数。

从这里我们可以看到，一个普通的 n 元函数可以通过以上的步骤，经过 n 次抽象，最终变为一个高阶函数。前面提到的 λ 演算的唯一一条函数定义方式就是抽象。

3.2 语法层面（形式定义）

从语用层面上，我们已经知道了一切函数都可以用匿名函数、柯里化和 λ -记号表示法表示为一种高阶匿名函数，并且我们可以将这种函数拿来计算。

这样我们就知道了形式系统 λ 演算中的一些表达式具体的含义，接下来，我们就要给出 λ 演算的一些形式定义，并且和其他的一些形式系统做对比，进而对 λ 演算和形式系统都有更深刻的认识。

3.2.1 符号字母表

λ 演算的符号字母表非常简单，仅仅包含三类符号：

$x, y \dots$	变量
λ	抽象符号
$., (,)$	标点符号

其中变量的集合是无穷字符串的集合。

3.2.2 λ 项

我们知道，在一阶谓词逻辑中，和函数对应（并不能说完全对应）的是项，而公式是和命题对应的。在 λ 演算中，我们想要做的是将函数及其变化形式化，所以在 λ 演算中，我们只需要定义项即可。

这里使用归纳定义来定义 λ 项：

- (1) 原子：变量是 λ 项。
- (2) 应用：若 M 和 N 是 λ 项，那么 (MN) 也是 λ 项。
- (3) 抽象：若 M 是 λ 项而 ϕ 是一个变量，那么 $(\lambda\phi.M)$ 也是 λ 项。
- (4) 限制：所有 λ 项的集合是由(1)(2)和(3)组成的。

举例来说，以下一些都是 λ 项： a 、 $((ab)c)d$ 、 $\lambda x.(yz)$ 、 $((\lambda y.y)(\lambda x.(yz)))$ 。

当然，在不产生歧义的情况下，我们当然也可以去掉一些不必要的括号，省略括号的规则如下：

- (1) λ 项中最外层的括号可以省略，如 $(\lambda x.x)$ 可以省略表示为 $\lambda x.x$ ；
- (2) 左结合的应用型的 λ 项，如 $((MN)P)Q$ ，括号可以省略，表示为 $MNPQ$ ；
- (3) 抽象型的 λ 项 $(\lambda\phi.M)$ 中，最外层的括号可以省略，如 $\lambda x.(yz)$ 可以省略为 $\lambda x.yz$ 。

3.2.4 语法全等

在 λ 演算中，我们也可以引入语法相等符号 $\equiv$ （当然，它不是 λ 项中的符号，而是元语言中的符号）。

语法全等可以表示为：

$$M \equiv N \quad (12)$$

表示 M 和 N 有完全相同的结构，且对应位置上的变量也完全相同。

语法全等的要求是很高的，实际上，在 λ 演算中还存在着其他形式的等价关

系，例如后面将要介绍的 α -等价。

3.2.5 自由变量与出现

为了后续的转变操作，我们也需要对 λ 项中出现的变量进行一定的分类。和一阶谓词逻辑等其他一些形式系统的做法一样，我们可以将 λ 项中出现的变量分为自由变量和约束变量。如果将 λ 演算和一阶谓词逻辑进行类比，我们就可以很清楚地理解自由变量和约束变量、自由出现和约束出现的概念。但是我们仍然可以形式化地定义自由变量和出现。

对于两个 λ 项 P 和 Q ，二元关系出现定义如下：

- (1) P 出现在 P 中。
- (2) 若 P 出现在 M 中或 N 中，则 P 出现在 (MN) 中。
- (3) 若 P 出现在 M 中或 $P \equiv \phi$ ，则 P 出现在 $(\lambda\phi.M)$ 中。

对于一个 λ 项 P ，定义 P 中自由变量的集合 $FV(P)$ 如下：

- (1) $FV(\phi) = \{\phi\}$
- (2) $FV(MN) = FV(M) \cup FV(N)$
- (3) $FV((\lambda\phi.M)) = FV(M) - \{\phi\}$ (集合的差运算)

(4) 除 (1) (2) (3) 步骤产生的 $FV(P)$ 中的元素外， $FV(P)$ 不包含其他元素。

所有在 P 中出现且不属于 $FV(P)$ 的变量称为约束变量。

如果对 λ 项 P ，有 $FV(P) = \emptyset$ (空集)，则称 P 是封闭的，或者称 P 是组合子。组合子的概念和一阶谓词逻辑中的闭公式有异曲同工之妙。一阶谓词逻辑中的闭公式对应于一个可以明确判断真假的具体的命题(前提是在某个特定的解释中)，而组合子对应于一个明确定义的可以通过变量代入求得一个具体解的函数(虽然这样说并不是完全准确)。

3.2.6 替换

只是简单地形式化定义出了 λ 项，并没有实际的用处，我们还需要定义一些操作，在 λ 中，很重要的一个操作就是替换。

从直观上来理解替换是很容易的，即是说将 λ 项中不被 $\lambda\phi$ 约束的变量 ϕ

替换成另一个λ项。我们可以简单地定义 $[N/\phi]M$ 是把M中出现的自由变量 ϕ 替换成N后得到的结果。

在实际操作中，这个替换可能会造成变量名的冲突（例如我们将 $(\lambda x.y)$ 变为 $(\lambda x.x)$ 就会产生麻烦），所以对于任意的λ项M、N和变量 ϕ ，定义替换：

- (1) $[N/\phi]\phi = N$
- (2) $[N/\phi]\alpha = \alpha$ （对所有不满足 $\alpha \equiv \phi$ 的原子 α ）
- (3) $[N/\phi](PQ) = ([N/\phi]P[N/\phi]Q)$
- (4) $[N/\phi](\lambda\phi.P) = \lambda\phi.P$
- (5) $[N/\phi](\lambda\phi.P) = \lambda\phi.P$ （若 $\phi \notin FV(P)$ ）
- (6) $[N/\phi](\lambda\phi.P) = \lambda\phi.[N/\phi]P$ （若 $\phi \in FV(P)$ 且 $\phi \notin FV(N)$ ）
- (7) $[N/\phi](\lambda\phi.P) = \lambda\eta.[N/\phi][\eta/\phi]P$ （若 $\phi \in FV(P)$ 且 $\phi \in FV(N)$ ）
- (8) 除了（1）-（7）之外，替换不包含其他的操作。

对其中从（5）-（7）的各条来说， $\phi \equiv \phi$ 不成立。对于（7）来说， η 是满足 $\eta \notin FV(NP)$ 的任意变量。

虽然看上去这个定义非常复杂，但是每一条其实都不难理解，因为替换的目的是把M中出现的自由变量 ϕ 替换成N，这么多条的作用就是为了避免变量名的冲突。

3.2.7 α-变换和α-等价

前面提到的λ演算只包括一条变换规则，说的就是α-变换。

下面给出α-变换的定义：

设 $\lambda\phi.M$ 出现在一个λ项P中，且设 $\phi \notin FV(M)$ ，那么我们将 $\lambda\phi.M$ 替换成

$$\lambda\phi.[\phi/\phi]M \quad (13)$$

的操作称为P的α-变换。

当且仅当P可以经过有限步（包括零步）α-变换，得到新的λ项Q，P与Q是α-等价的，又写作：

$$P \equiv_{\alpha} Q \quad (14)$$

我们看到，α-等价相较于语法全等，要求要低得多，但是它依然有些重要的性质。可以证明， $\equiv_{\alpha}$ 满足自反性、对称性和传递性。

α -变换说的是什么呢？我们可以利用一个例子来直观的理解：

$$\lambda x.\lambda y.x(xy) \equiv_{\alpha} \lambda x.\lambda v.x(xv) \equiv_{\alpha} \lambda u.\lambda v.u(uv) \quad (15)$$

我们可以用普通的函数，例如 $f(x, y) = x + xy$ 来模拟这个过程，可以有如下的变换（我们暂且用 $\equiv$ 来表示这个过程）：

$$f(x, y) = x + xy \equiv f(x, v) = x + xv \equiv f(u, v) = u + uv \quad (16)$$

可以看到， α -变换实际上就对应着函数中的变量换名，并没有改变原式本质上的含义（可以这么认为）。

在其他的形式系统中，也有类似的对替换（或者说置换）的定义，并且也有着类似的等价关系，例如在命题逻辑中的置换定理：

设 $\Phi(A)$ 是含形式 A 的命题形式， $\Phi(B)$ 是用形式 B 置换 $\Phi(A)$ 的一次或多次出现而得到的命题形式，若 $B \Leftrightarrow A$ （ $\Leftrightarrow$ 也是一种等价的记法），则 $\Phi(B) \Leftrightarrow \Phi(A)$ 。

[6]18

3.2.8 β -规约

β -规约是对变换的一种限定，即是说，规约不能改变表达式的含义。当然，在 λ 演算中，不止存在着 β -规约这一种规约方式，还有其他的一些规约，例如 η -规约等。

我们定义形如

$$(\lambda\phi.M)N \quad (17)$$

的项为 β 可约式，对应的项

$$[N/\phi]M \quad (18)$$

称为 β 缩减项。

当 λ 项 P 中含有 β 可约式 $(\lambda\phi.M)N$ 时，我们可以把 P 中的 $(\lambda\phi.M)N$ 整体替换成 $[N/\phi]M$ ，用 R 指称替换后得到的项，那么我们说 P 被 β 缩减为 R ，写作：

$$P \triangleright_{1\beta} R \quad (19)$$

当 P 可以经过有限步（包括零步）的 β 缩减后得到 Q ，则称 P 被规约到 Q ，写做：

$$P \triangleright_{\beta} Q \quad (20)$$

我们也可以用普通的匿名函数来模拟这个过程（暂且用 $\triangleright$ 来表示这个过程），

例如：

$$(x \mapsto 2x)3 \triangleright 2 \times 3 = 6 \quad (21)$$

这就相当于普通函数的如下过程：

$$f(x) = 2x \quad (22)$$

$$f(3) = 2 \times 3 = 6 \quad (23)$$

这样我们就可以看到，β 缩减的过程其实就对应于函数调用过程，即代入过程。

还有一点需要注意的是，规约的顺序并不影响最终的结果，例如（注意，这里使用的不是纯逻辑）：

$$(\lambda x.((\lambda y.(x+y))v))u \triangleright_{1\beta} \lambda y.(u+y)v \triangleright_{1\beta} u+v \quad (24)$$

$$(\lambda x.((\lambda y.(x+y))v))u \triangleright_{1\beta} (\lambda x.(x+v))u \triangleright_{1\beta} u+v \quad (25)$$

在其他形式系统中，也有类似的对代入的定义，目的也在于不改变表达式的含义下进行变换。例如命题逻辑中的代入定理：

设 A 是一命题形式，在其中出现的命题变元是 $p_1, p_2, \dots, p_n$ ，又设 $A_1, A_2, \dots, A_n$ 是任意命题形式，若 A 是重言式，则用 $A_i (i=1, 2, \dots, n)$ 代换 A 中 p_i 的每个出现得到的命题形式 B 也是重言式。^{[6]17}

虽然在命题逻辑中，这样做的目的是得到一个新的重言式，但是两者有着类似的思想。

于是，通过以上的一些形式定义，一切关于函数的操作（其实应该说是可计算函数）我们都可以通过 λ 演算这个形式系统来完成，也就是说，但这里为止，我们就已经给出了 λ 演算的完整的定义，虽然关于 λ 演算，还有很多很多的其他相关定义，但我们暂时只需要定义到这里就可以了。^⑬

3.3 λ 演算与其他形式系统的比较

3.2 中对 λ 演算的形式化定义已经很完整了。对于一个形式系统，我们当然会想知道它的能力到底如何，这时候，一个比较直观的方法就是把它和其他一些形式系统做对比。

^⑬ 3.2 节主要参考了参考文献列表中的[7]，因为引用的地方比较多，不再一一标注。

3.3.1 λ 演算与一阶谓词逻辑

虽然使用 λ 演算思想的 Lisp 诞生的很早，但是在种种因素的共同作用下，这门被称为“神之语言”的编程语言却被类 C 语言逐步淹没。这就说明相比于 λ 演算，其他的一些形式系统一定有它更好用的地方。

在其他的形式系统中，我们主要关注谓词逻辑。

谓词逻辑^⑭是一种基于谓词分析的高度形式化的语言，是人工智能科学赖以产生和发展的最古来、最直接，也是最完备的理论基础。^[8]虽然高阶的谓词逻辑看起来拥有更强的能力，但是在实际应用中，应用最广的还是一阶谓词逻辑。

在上一章中已经提到过了本体论假设，如果本体论假设成立，结合一阶谓词逻辑，我们就可以对整个现实世界进行表征和计算，事实上，一阶谓词的应用确实让逻辑符号主义学派的人工智能红极一时。

但是本体论假设很可能是不成立的^⑮，而且，随着编程语言的发展，越来越多的人开始注意到函数式编程的优势。诚然，我们现在的很多程序员和算法设计师并不会首先考虑逻辑符号的方法，像神经网络这样的算法也很难从中寻找到逻辑，对于现代 IT 产业相关的人们来说，程序是什么呢？是函数，全都是函数！

有了上面一些结论，我们就可以窥探到，相比于一阶谓词逻辑， λ 演算有着不可替代的优势：

(1) 虽然从理论上来讲，一阶谓词逻辑有着很强的表达能力，它具有自然性、精确性、容易实现等等优势^⑯，但是在有些时候，直观上看起来，它的表达能力并不能使我们满意^{[1]⑰}，而 λ 演算为任何函数冠以复杂而又有规则的省缺名，

^⑭ 如果不特别说明，本文中类似于“XXX 逻辑”的表述都是指的一个形式系统。

^⑮ 德雷福斯的人工智能批评理论中对这一点进行了说明。

^⑯ 这里暂且不谈一阶谓词逻辑无法表示不确定性知识这一点，因为在一阶谓词逻辑和 λ 演算的比较中，这一点是没有意义的。

^⑰ 关于这一条表述，我只是引用了参考文献中的原文，但是我个人感觉这句话说得不够清楚（当然，这三条比较中也都有我自己通过查阅资料或者举例而得到的一些理解），但是目前还没有其他相关表述或者证明。

于是就可以方便的表达所有函数。

(2) λ 演算的转换规则的操作方式都是等式方式，所以自动推理过程的效率很高，并且具有“可运算”的精确定义。一阶逻辑形式过于灵活，这也使得它过于复杂，自动推理过程主要依靠搜索、匹配、合一、消解，效率从理论上难以提高。

(3) 一阶谓词逻辑并没有直接体现出计算能力。在一阶谓词逻辑中，在一个确定的解释中，一个闭公式^⑩的取值只有真和假，或者说 0 和 1，对于函数的计算也要想办法转换成命题之间的推导关系来进行处理，非常麻烦，往往无法直接用于计算。而 λ 演算的基本计算手段就是归约，前面也提到了，公约就相当于函数调用或者代入， λ 演算直接体现出了计算能力。

3.3.2 λ 演算与元胞自动机

既然本学期也学习了元胞自动机这种形式系统，我们也可以把元胞自动机和 λ 演算进行一些对比。

元胞自动机的一个很大的优势就在于它的规则是很方便人来理解的，而要理解 λ 演算的思想，就需要 3.1 和 3.2 节中的一些先验知识。

要定义一个元胞自动机也很简单（毕竟也是形式系统嘛），只需要初始状态和规则就可以了^⑪，接下来的事情就可以自动的进行下去，进而产生很多可以观察到的有趣的现象，例如可以用元胞自动机产生一些周期性变化的图形、兰顿蚂蚁等现象，但是有些元胞自动机却被归类为混沌型或者是复杂型，我们很难从中找到明确的规律。^[9]

这说明，元胞自动机是非常依赖于外在的表征和解释的。对于像图灵机这样的比较看重操作的形式系统也是一样，都需要外在的表征作为初始状态，并且需要对结果进行解释才可以发挥实际的作用。而对于 λ 演算来说，所有的操作步骤都直接对应于函数的操作，表征和解释非常容易。

^⑩ 为了方便讨论，这里只谈论闭公式和组合子。

^⑪ 符号等其他构成要素对我们这里的讨论没有太大关系，所以这里可以不去谈论它们。

3.3.3 λ 演算与其他一些有关可计算函数的形式系统

但是我们需要看到， λ 演算和命题逻辑、一阶谓词逻辑的一个很重要的不同就在于 λ 演算主要是为了可计算函数而被设计出来的，而命题逻辑和一阶谓词逻辑主要是为了研究命题而被设计出来的，两者的区别当然会很多。既然 λ 演算主要是为了函数而被设计出来的，我们当然可以拿它和其他一些有关函数的形式系统做出比较。

在做出比较之前，我们先对命题逻辑和与其等价的一种变体——形式系统 L 做比较研究。

我们可以简单的解释一下命题逻辑和形式系统 L 的关系：在命题逻辑中，存在有五个联结词 $\{\wedge, \vee, \leftrightarrow, \rightarrow, \sim\}$ （不同的逻辑书中，表达这五个联结词使用的符号可能会有所差别，本文中统一使用这五个符号），但是，我们可以证明 $\{\rightarrow, \sim\}$ 是联结词的完备集，所以所有命题逻辑中的公式就改可以写成仅用 $\rightarrow$ 和 $\sim$ 两个联结词表示的公式，在形式系统 L 中，公式就是这样表示的。

命题逻辑主要有 11 条形式推演规则，而对于形式系统 L，只存在有三条公理和一条形式推演规则。（这里不考虑一些二级推论）

我们可以简单地考察命题逻辑中的两条形式推演规则[]：

(3) 如果 $\Sigma, \sim A \vdash B$;

$\Sigma, \sim A \vdash \sim B$;

则 $\Sigma \vdash A$

(反证律，或者 $\sim-$)

(7) 如果 $\Sigma \vdash A$;

$\Sigma \vdash B$;

则 $\Sigma \vdash A \wedge B$

($\wedge+$)

对于 (3)，我们可以借用数学中的反证法来很好的理解，对于 (7)，我们也可以借助（非形式化的）命题逻辑中的基本规律来理解，也就是说，对于命题逻辑，我们是比较清楚每一步在做什么的^⑩，命题逻辑的抽象程度并不高，形式推演规则却比较多，实际进行形式推演时要找到正确的规则并不容易。

^⑩ 虽然实际上对于形式系统来说，操作本身是不“附加”意义的，一切只不过是字符串的变换操作而已，但我们依然用被形式化之前的系统的操作来和形式系统的一些操作进行对应。

接下来我们考察形式系统 L 的一条公式^[6]:

$$(L2) (A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C))$$

对于 L2, 我们并不能马上识别出这个公理表达的意思。对于其他两个公理也是一样的。前面已经提到, 形式系统 L 中只有两个联结词 $\rightarrow$ 和 $\sim$, 所以形式系统 L 是比命题逻辑更加抽象的形式系统, 我们要理解每一步操作的含义并不容易, 但是形式系统 L 的公理和规则比命题逻辑要少很多, 实际应用起来要更加高效。

这样, 我们就可以发现, 一般来说, 一个形式系统的抽象程度越高, 它就越高效。

λ 演算的抽象程度是很高的, 以至于它只包括一条变换规则和一条函数定义方式, 所以, 相比于其他一些有关可计算函数的形式系统来说, λ 演算往往是更加高效的。

4 λ 演算的应用初探

虽然我并不想过多地谈及 λ 演算的应用，因为这样做的话基本上就不得不谈到函数式编程，这不是本文研究的重点，但在听完海健的报告后，我觉得还是可以把他的知识也总结进来，也使得我们不会总是从一个抽象的层面上来理解 λ 演算。

λ 演算当然不是被单纯的构造出来折磨人的，它是具体的和可操作的。

4.1 函数式编程

谈到 λ 演算的应用，自然少不了对函数式编程的讨论。

4.1.1 什么是函数式编程

函数式编程主要是相对于命令式编程²¹来说的。

命令式编程是当前使用最广泛的编程方式，它要求我们要告诉计算机每一步要做什么，即是说，我们要用一系列的命令来告诉计算机指令的执行顺序，这些命令包括直接和硬件相关的命令，还有一些经过封装的命令。

函数式编程的概念于 1978 年提出并且是由 λ 演算演化而来的。

函数式编程的最核心的思想就是把函数看成是系统中的“一等公民”，即是说，函数可以被当成值来任意进行传递。

在命令式编程里，我们也不是没有办法将函数作为一个参数传入另一个函数，例如 C 语言中我们可以使用函数指针，在 MatLab 中我们可以使用函数句柄等，但在命令式编程的世界中，函数毕竟不是“一等公民”，函数和变量的地位是不一致的。命令式编程做不到函数式编程的那种简洁。

函数式编程的关注点不在于底层实现，而是侧重于程序该做什么（而不是命令式编程强调的怎么做）。

需要说明的一点是，实际应用中，函数式编程也不得不考虑底层实现，也要

²¹ 除了命令式编程、函数式编程之外，还有其他的编程范式，例如声名式编程（SQL 就是用这种方式），为方便起见，这里只讨论命令式编程和函数式编程。

考虑 I/O 操作，函数式编程只是要试图将这些操作限制到最小。

4.1.2 函数式编程的优势与不足

函数式编程有很多命令式编程没有的优势^[10]。

(1) 代码简洁、开发迅速。函数式编程大量使用函数，减少了代码的重复，因此程序比较短，开发速度较快。

(2) 更接近自然语言。函数式编程的自由度很高，可以写出很接近自然语言的代码。

(3) 易于“并发编程”。函数式编程不会修改变量，因为就不会发生“死锁”的现象，用函数式编程写多线程程序要比命令式编程少考虑很多东西。

但是目前使用最广的依然是命令式编程，说明函数式编程也存在一些不足，除去复杂的历史原因外，普遍认为函数式编程严重耗费 CPU 和存储器资源，主要原因有两个：

(1) 早期的函数式编程语言实现时没有考虑过效率问题。

(2) 有些非函数式编程语言为求提升速度，不提供自动边界检查或自动垃圾回收等功能。

关于函数式的具体内容在这里不进行论述。

4.2 λ 演算的变体

λ 演算还有一些变体，这些变体也被应用于一些编程语言之中。

4.2.1 π 演算

π 演算是英国人米勒 (R. milner) 教授与其合作者基于 λ 演算、顺序范型和计算机交互式并发通讯操作系统，于 1993 年提出的一个全新的基本并发演算。

π 演算属于进程代数类演算。

π 演算的核心思想是名调用；其关键技术是命名和参考。其他语言中的各种变量、函数、甚至是进程，在 π 演算这里都可看作用项表示的名，有关的计算都可通过名的调用和名的计算来实现。^[1]

4.2.2 χ 演算

虽然 π 演算适合于交互式并发通讯, 但是 π 演算本身也存在有很多问题, 即使是经过改进的高阶 π 演算也没有解决这些问题。 π 演算的表达能力也没有 λ 演算强。

为了找到一个可以完全模拟 λ 演算操作语义的并发计算模型。傅育熙提出了 χ 演算。 χ 演算对 π 演算进行了简化, 但是其表达功能却比 π 演算强, 高阶 χ 演算可以完全模拟 λ 演算的操作语义。

并且, 我们可以证明, 使用 π 演算的 π 进程是使用 χ 演算的 χ 进程的子进程, π 语言是 χ 语言的一个子语言。^[11]

5 形式系统、语言、世界与计算

本章的内容可能有些激进。本章试图达成某个比较宏伟的目的，即更好的理解维特根斯坦的理论。在我看来，维特根斯坦前期的理论和后期的理论之所有有很大的差异，很重要的一点是他看待形式系统与语言的态度发生了变化，这造成了他对现实世界认识的转折。

本章主要基于维特根斯坦前期和后期的理论，集合我在本科阶段学习到的一些知识和上面几章的内容，讨论四个重要的概念：形式系统、语言、世界与计算。

5.1 维特根斯坦是谁

前文虽然多次提到了维特根斯坦，但是却还没有正式地介绍一下他。

路德维希·维特根斯坦（Ludwig Wittgenstein, 1889 年 4 月 26 日—1951 年 4 月 29 日）是分析哲学的创始人之一，也是位具有传奇色彩的哲学家。如果说只研究康德就可以养活一批哲学家的话，那么只研究维特根斯坦就可以养活三批哲学家。之所以这么说，是因为看上去，维特根斯坦前期和后期的哲学理论几乎是完全不同的²²，并且，前期与后期的理论都不好理解。

要完整的阐述维特根斯坦的思想当然非常困难，但是我们仍然可以从中窥探到一些东西。

特别需要注意的一点是，维特根斯坦一生的哲学追求可能只为了回答一个问题：我们可以认识真理吗？

5.2 维特根斯坦前期理论、形式系统与世界

维特根斯坦前期理论主要体现在它的著作《逻辑哲学论》中。《逻辑哲学论》的整体写作风格非常有条理性，全书分为 8 个大命题，大部分大命题下面都有对这个大命题解释的子命题，子命题下又有子子命题，形成一个三级结构。这种写

²² 之所以说研究维特根斯坦就可以养活三批人是因为我们可以大体上把维特根斯坦的哲学分为前期和后期，因为前期和后期维特根斯坦的理论看起来完全不同，所以也有很多人试图研究维特根斯坦思想的变化过程，即所谓的“维特根斯坦中期思想”。

法与其说是写成了一步作品，更像是定义出了一个完整的系统。前期的维特根斯坦对自己的理论很有自信，认为通过《逻辑哲学论》，他已经用这本书将哲学的全部解释清楚了。

整个《逻辑哲学论》其实是建立在一个极强的逻辑主义的思想之上的，我们可以考察其中的几个大命题：

- (1) 世界是所有发生的事情。
- (3) 事实的逻辑图像就是思想。
- (4) 思想是有意义的命题。
- (7) 对于是不可言说的东西，我们必须对之保持沉默。^[12]

我们可以看到，在《逻辑哲学论》中，事实、思想和命题有着极强的一一对应关系，这和本体论假设的思想是一致的，整个《逻辑哲学论》中就在做一种表征工作，将世界图示化地表达出来。简要概括《逻辑哲学论》的思想，即世界的逻辑构成和逻辑图像论理论。

整个理论中，比较特别的是命题（7），他没有任何子命题对它进行进一步地说明，或者正如命题本身所描述的那样，我们没法用语言来对不可言说之物进行任何说明。

但是命题（7）是必不可少的，有了它，我们才能较为完整地阐述维特根斯坦前期思想：世界分为可以言说的部分和不可以言说的部分，可以言说的部分必然是符合逻辑的（我们只能逻辑的思考，一切能够思考的东西才可以在语言中得到相应的表达），所以，“我的世界”²³总已经是一个被逻辑地思考了或正在被逻辑地思考着的世界，“我的世界”中不可能存在不可以被形式化表达的东西。当然，对于这些存在，我们也不是完全没有办法处理他们，虽然它们无法被言说，但我们仍有方法在“我的世界”中显示它们。

或者更简单来说，“我的世界”是被我的语言和思维限定的世界，而语言和思维都是必然符合逻辑的，“我的世界”是有逻辑构成的世界。

²³ 这里可以简单地说明一下，在哲学界，有“我的世界”这样的表述，“我的世界”和我们平常认知的科学定义的客观存在的世界不同，意思更接近我们可以直接体验到的“实事”的总和，是主观意义上的世界，是一个更加抽象的概念。

这里暂且不谈语言的问题，因为后期维特根斯坦对语言的理解发生了很大变化²⁴。我们发现，维特根斯坦构建的“我的世界”是由纯粹逻辑构成的世界，是可以形式化地表达出来的。

也就是说，我们其实可以把形式系统中一切的集合看作是一个世界，这是一个纯粹由符号、公理和规则规定的世界。并且，按照维特根斯坦前期思想的说法，这个世界是和“我的世界”中的存在是一一对应的。

5.3 维特根斯坦后期理论、语言与世界

维特根斯坦的《逻辑哲学论》虽然将“我的世界”解释成一个由语言限定的纯粹逻辑的世界，并且看起来命题（7）用一个非常巧妙的方式避免了真实世界中那些不符合逻辑的部分，但后来，这一条命题可以说是渐渐改变了维特根斯坦的想法。

后期维特根斯坦的思想更多的是像杂记一样，更多的像是一些哲学体悟，虽然我们仍然可以从中总结出维特根斯坦的一些思想。

在后期维特根斯坦的思想中，很重要的一个思想就是“遵守规则”悖论。在《逻辑哲学论》构建的“我的世界”中，一切都是符合逻辑的，都是遵守规则的，但是事情好像不可以从形式语言扩展到自然语言，自然语言是遵守规则的吗？也许很多人会说，我们确实会学了很多语法啊，语法难道不是语言的规则吗？其实并不是，因为语法也只是我们根据语言的实际使用总结出来的，我们的语言也没有必要完全遵守规则，并且，语法作为元语言，和对象语言是同一种语言（这里暂且不谈用中文解释英文语法的情况）。也就是说，规则其实是在生活活动中形成的，而不是分析、解释出来的²⁵。

后期维特根斯坦的思想另一个重要的思想是否定了私人语言的存在，这一点非常重要。前期维特根斯坦认为语言是“我的世界”的限制，但这有个问题，现实中，我们使用自然语言，但这个语言是公共的还是私有的？后期维特根斯坦认

²⁴ 其实在这里，维特根斯坦说的“语言”完全可以等价于形式语言，这将在后面说明。

²⁵ 这也是为什么后期维特根斯坦放弃了《逻辑哲学论》的写作模式，因为我们并不能用这些“人为”总结出来的规则来解释世界。

为语言是公共的，这一点直接说明了每个人的“我的世界”其实都是一个同一个世界，人与人之间是完全可以相互理解的。

后期维特根斯坦的思想中还有比较重要的是语言游戏论。简单来说，语言游戏是一种活动，语词的意义和它的使用规则根植在人们的生活形式中，一个语词在实践中的用法就是它的意义。

当然，后期维特根斯坦的思想还有很多非常重要的理论，例如家族相似等，这里不再详细地谈论。

当然，后期维特根斯坦的思想也有很多不足和偏激的地方，它的哲学最终使他走向了反哲学的立场。他认为我们至今为止构建出的哲学大厦都只是语言游戏的产物，我们的世界本来就是由语言构成的完备的世界，哲学本身并没有创造任何新的东西，反而将事情变得更加复杂，阻碍了我们的认知²⁶。

不管怎么说，我们仍然可以从后期维特根斯坦的思想中看出语言和世界的关系。在《逻辑哲学论》中，语言其实是思想和世界的一种对应关系，是对世界的一种映射关系。而在后期维特根斯坦这里，语言是语言游戏的聚合，它植根于人们的生活形式之中。

当然，没有变化的一点是，语言仍然是“我的世界”的界限，是我们的认知的界限，“我的世界”依然是由语言限定的，不过语言已经不满足前期维特根斯坦的强逻辑假设了。

5.4 计算

作为本文理论部分的最后几节（后两章是回答问题和总结），当然要回到非经典计算课程的一大主题——理解计算的本质。我们已经学过很多经典计算的理论，本学期也学到了很多非经典计算的理论，并且，前文更是从更高的、更抽象的哲学层次来探究了形式系统、语言与世界的关系。这样，我们就可以重新审视“计算”。

²⁶ 这个说法其实是有事实依据的，因为每当哲学发展陷入危机而爆发哲学革命时，解决的途径往往是退回到之前一些哲学家的思想上，试图对他的哲学进行新的解释，最常见的就是回到柏拉图那里，回到哲学的起点，这看起来就像是回归了哲学原本的样子，而每一次对原本哲学的解释好像都必然走进了死路。

在对计算进行说明之前，我们先来了解一下 19 世纪末哲学革命产生的两大新的哲学派别中的另一条分支——现象学，这里简要谈一下现象学创始人胡塞尔（Husserl）的现象学。

现象学是强调“回到实事²⁷本身”的一门哲学，与分析哲学这种排除心理学和认识论因素的哲学不同，现象学的产生本来就深受格式塔心理学的影响。

胡塞尔强调我们要对客观事物进行还原，悬搁，来获得只存在于我们的思维之中的纯粹的内容。胡塞尔提出纯粹意识，或者说先验自我的概念。他认为先验自我是世界中的一切对象显现的先验条件。在这些对象显现的过程中，总伴随着某种理解和某种意义的生成，世界是先验自我²⁸的构成物^[13]。

即是说，胡塞尔将发展了理智主义，将理智主义的思想基础全部归结为意识，意识可以做到把一切存在都看成是确定性的事物。并且胡塞尔将意识的地位提高到极高的高度。

或者说，胡塞尔将意识看做是意义赋予的机制。

考察形式系统，形式系统本身的操作是不附带任何意义的。课堂上讲到，计算的本质就是物理过程、就是变化。也即是说，广义上的计算本身其实也是没有附带意义的，只是遵循着物理规则的过程。意义是我们人为赋予的。

但是我认为，计算仍然不止局限于物理过程。上述规定的计算本身并没有意义，但是人类是追求意义的生物，在胡塞尔这里，意识就是这种意义赋予机制，它赋予计算的初始条件、计算过程和计算结果以意义。之前我们谈到用形式系统解决一个现实问题的步骤，即表征、形式推演和解释，其实就是联系计算和有意义的世界之间的桥梁。

当然，我们可以用维特根斯坦的语言来替换掉胡塞尔的“意识”，即是说，语言是赋予意义的工具。虽然维特根斯坦的家族相似理论认为我们并不能用统一的定义来说明语言游戏的本质，但是我们依然看到，这些语言游戏本质上也都是

²⁷ 实事并不是所谓纯粹客观的事实、事件，而是直观中被直接给予的东西（感觉材料和对象；感性的和理智的），也即哲学所探讨的问题本身。

²⁸ 简单理解来说，先验自我就是最主观的自我，它作为一切的背景，无法被其他任何事物所解释。一切事物都必须以它作为背景才可以获得意义。

在做意义赋予工作。

这里，还有另外一个问题要解决：意义赋予是一种计算吗？²⁹我的观点是：这也是一种计算，虽然意义赋予不能被简单的理解成一种映射，但是其处理过程其实也可以看作是一种变化。

我们可以大胆的得出结论：一切过程都是计算。我们甚至可以将计算和马克思主义哲学结合起来，即是说，马克思主义哲学中的“运动”就是我们所说的计算。

从这点上来说，我也是赞成所谓的“计算宇宙”的论点的。除了上述定义的客观上的运算外，我们也可以定义主观上的运算。主观上的运算是被语言限定并进行意义赋予的特殊的运算，或者说，计算即智能，或者说，计算就是一切智能过程。

我所得出的这个结论也是一个所谓的强结论。但是，和老师这学期给我们的《计算理论教学中的一些逻辑思考》中“计算极限=数学极限=智慧极限”[]理论不同，这里我想要阐述的计算是一种更广泛意义上的计算，换言之，不将计算定义地那么广，这个论点是很难成立的。

5.5 形式系统、语言、世界与计算

这样，我们就来总结一下形式系统、语言、世界与计算四者之间的关系。

形式系统是形式语言对应的系统，自然语言可以作为形式语言的元语言而存在³⁰。形式系统是遵循规则的，并且这个规则是通过其元语言规定的。自然语言，或者说语言，并不一定要是遵循规则的，特别对于自然语言来说，规则其实是在生活活动中形成的，而不是分析、解释出来的。

²⁹ 这里还存在着有一个相关的重要问题，即我们大脑中的所有活动是不是都是可以还原成物理过程的，如果我们对这个问题给出肯定的答案，那么接下来的讨论将会是没有意义的，因为既然意义赋予系统已经是物理系统了，当然其中的一切操作也都是物理过程，也可以看成是计算。事实上，现在哲学家和认知科学家认为大脑中的所有活动不能全部还原为物理过程（当然，这可能也是因为最近非理性主义抬头造成的结果），这里就采用这种观点。

³⁰ 当然，我们也可以用形式语言来定义另一个形式语言。

在形式系统中，计算就是所谓字符串的变化，即对规则的使用。在真实世界中，自然语言有着和形式语言类似的功能，课堂上提到的计算，客观上是物理过程，但要使得计算的初始条件、计算过程和计算结果有意义，就要借助于语言，虽然语言并不总是“遵循规则”的。

基于上述分析，我们可以给出新的对计算的定义，即客观上，一切过程都是计算；主观上，一切智能过程都是计算。

6 对于报告问题的回应

本章主要对课堂上同学们提出的一些问题做出回答。

(1) 请问表征真的那么重要吗？

回答：人工智能的主导范式是认知主义。它是符号主义、功能主义、计算主义与表征主义的集成，而它的核心概念是表征。“表征就是用来加工的信息束。像知觉和注意这样的认知加工，就是去解码来自世界的信息束，并创造或改变我们的表征。推理和决策过程就是表征加工，从而形成新信念和特定行动。加工就是对信息的动态运用。表征就是可用的信息。”也就是说，人工智能研究的主要方法是从现实世界中抽象出模型与获取信息，并用这个模型去处理信息，得到结果。

7 写在后面的话

虽然本文并不是以得出某些具体的结论为目的的，但在这个过程中，我依然有很多想说的话，也算是对本文的一个小小的总结吧。

这里，我们或许再次可以回到前言中提到那个问题：为什么我总是喜欢去研究一些比较老的问题？

结合之前我们研究过的很多话题，我们可以换一个角度来回答这个问题。人工智能领域有两大阵营——强人工智能主义和弱人工智能主义。相对来说，我们很容易对强人工智能主义进行反思和批评，因为他们提出的假设往往都太强了，要满足这些假设其实是很难的，所以我们并不需要过多的先验知识就可以对强人工智能主义进行讨论。但对于弱人工智能主义，我们却难以对其进行反思，更不要说提出一针见血的批评意见了，因为看起来弱人工智能主义是向现实妥协的。

要对弱人工智能进行反思，我们必须要有足够多的知识储备，要对数理逻辑、计算机科学、认知科学等多个领域有所研究。

我觉得通过对形式系统，特别是对 λ 演算的研究可以更好地帮助我对人工智能有一个更深入的理解。虽然本文讨论的很多话题看起来和 λ 演算本身的关系并不是特别密切，这也是因为我对 λ 演算本身的理解其实还没有到位，并且，我的整个理论还没有成熟，还没有做到完全将 λ 演算本身也加入到一些解释之中。

当然，就本文来说，最终目标是试图对形式系统、语言、计算和世界的本质有更深入的认识，并且理清这些概念之间的联系。

我们在课堂上学习到了很多非经典计算，这些非经典计算理论也旨在弥补经典计算的不足，我坚信人工智能的前景是光明的。

最后还是要回到一个更加根本性的问题，我们为什么要研究人工智能，为什么要从更高的层次对人工智能不断提出批评意见？我的回答是：人工智能的发展并不是要取代人的智能，最终的目的还是为了理解人的智能。也正如德雷福斯说：“的确，最近报道的计算机使用的成功，扩大了人的智能而不是取代了人的智能”^{[2]308}。为了更好地研究人的智能，我们当然根据实践经验和相关理论对人工智能这个工具进行不断地优化改进，以便使它更好地发挥作用。

参考文献

- [1] <https://recomm.cnblogs.com/blogpost/7080876>
- [2] 休伯特·德雷福斯. 计算机不能做什么——人工智能的极限[M]. 生活·读书·新知 三联书店:北京, 1986:14-308.
- [3] 赵小军. 计算主义形式系统难题:基于哥德尔不完全性定理的讨论[J]. 洛阳师范学院学报, 2018, 37(07):12-19.
- [4] 何红英. 从欧几里得《几何原本》谈公理化思想[J]. 西安文理学院学报(自然科学版), 2018, 21(06):11-13.
- [5] 罗杰·彭罗斯. 皇帝新脑[M]. 湖南科学技术出版社:湖南, 1995:453-466.
- [6] A. G. 哈密尔顿. 数学家的逻辑[M]. 商务印书馆:北京, 1989:17-18.
- [7] 王盛颐. 让我们谈谈 λ 演算[Z].
- [8] 刘素姣. 一阶谓词逻辑在人工智能中的应用[D]. 河南大学, 2004.
- [9] 课程课件
- [10] 王学瑞. 函数式编程语言发展及应用[J]. 计算机光盘软件与应用, 2012, 15(23):181-182.
- [11] 徐林, 傅育熙. λ -演算与 π -演算的语义比较研究[J]. 计算机科学, 2000(02):12-15.
- [12] Ludwig Wittgenstein. Logisch-philosophische Abhandlung[M]. Tractatus Logico-Philosophicus:London, 1922:9-111.
- [13] 赵敦华. 现代西方哲学新编(第二版)[M]. 北京大学出版社:北京, 2001:95-102.

附录 A： 希尔伯特问题与解决现状

主要参考资料：<https://www.cnblogs.com/wendcn/p/12606124.html>。

希尔伯特问题比较重要，其中的有些问题比较具体，有的则比较抽象。这里简要列举一下这 23 个问题和他们各自的解决现状：

(1) 连续统假设问题。1963 年，Paul J. Cohen 在下述意义下证明了第一个问题是不可解的。即连续统假设的真伪不可能在 Zermelo-Fraenkel 公理系统内判定。

(2) 算术公理系统的无矛盾性（或相容性）问题。1931 年哥德尔的不完备性定理指出了用“元数学”证明算术公理的相容性之不可能。数学的相容性问题至今未解决。

(3) 两等高等底的四面体体积之相等问题。1900 年即由希尔伯特的学生 M. Dehn 对这个问题给出了肯定的解答。

(4) 直线作为两点间最短距离问题。希尔伯特之后，许多数学家致力于构造和探索各种特殊的度量几何，在研究第四问题上取得很大进展，但问题并未完全解决。

(5) 不要定义群的函数的可微性假设的李群概念问题。这个问题于 1952 年由 Gleason, Montgomery, Zipping 等人最后解决，答案是肯定的。

(6) 物理公理的数学处理。在量子力学、热力学等领域，公理化方法已获得很大成功，但一般地说，公理化的物理意味着什么，仍是需要探讨的问题。概率论的公理化已由 A. H. Konmoropob 等人建立。

(7) 某些数的无理性与超越性问题。1934 年 A. O. temohm 和 Schneieder 各自独立地解决了这问题的后半部分。

(8) 素数问题。一般情况下的黎曼猜想至今仍是猜想（2018 年，迈克尔·阿蒂亚爵士曾经给出过黎曼猜想的证明，但是是否正确还有待确认）。包括在第八问题中的 Goldbach 问题至今也未解决。

(9) 任意数域中最一般的互反律之证明。该问题已由高木贞治和 E. Artin 解决。

(10) Diophantius 方程可解性的判别问题。1970 年由苏、美数学家证明希尔伯特所期望的一般算法是不存在的。

(11) 系数为任意代数数的二次型问题。H. Hasse 和 C. L. Siegel 在这问题上获得了重要的结果。

(12) Abel 域上 kroneker 定理推广到任意代数有理域。尚未解决。

(13) 不可能用只有两个变数的函数解一般的七次方程。连续函数情形于 1957 年由苏数学家否定解决，如要求是解析函数，则问题仍未解决。

(14) 证明某类完全函数系的有限性问题。1958 年永田雅宜给出了否定解决。

(15) Schubert 记数演算的严格基础问题。Schubert 演算的基础的纯代数处理已有可能，但 Schubert 演算的合理性仍待解决。至于代数几何的基础，已由 B. L. Vander Waerden 与 A. Weil 建立。

(16) 代数曲线与曲面的拓扑问题。尚未解决但问题的前半部分有很多突破。

(17) 正定形式的平方表示式问题。该问题由 Artin 于 1926 年解决。

(18) 由全等多面体构造空间。部分解决。

(19) 正则变分问题的解是否一定解析问题。该问题在某种意义上已获解决。

(20) 一般边值问题。尚未解决。

(21) 具有给定单值群的线性偏微分方程的存在性问题。已由希尔伯特本人 (1905) 年和 H. Rohrl (德, 1957) 解决。

(22) 解析关系的单值化。一个变数的情形已由 P. Koebe (德, 1907) 解决。

(23) 变分法的进一步发展。希尔伯特本人和许多数学家对变分法的发展作出了重要的贡献。

附录 B: 关于命题逻辑的一些形式定义

定义 1: 符号字母表

$p, q, r, \dots$	命题符号
$\sim, \wedge, \vee, \rightarrow, \leftrightarrow$	联结符号
$(,)$	标点符号

定义 2: 原子公式

命题符号即为原子公式。

定义 3: 公式

- (1) 每个原子公式都是公式。
- (2) 如果 a 和 b 是公式, 则 $(\sim a)$, $(a \wedge b)$, $(a \vee b)$, $(a \rightarrow b)$ 和 $(a \leftrightarrow b)$ 也是公式。
- (3) 只有通过以上规则得到的表达式才是公式。

定义 4: 形式推演规则

- (1) $A \vdash A$ (Ref.)
- (2) 如果 $\Sigma \vdash A$;
则 $\Sigma, \Sigma' \vdash A$; (+)
- (3) 如果 $\Sigma, \sim A \vdash B$;
则 $\Sigma, \sim A \vdash \sim B$;
则 $\Sigma \vdash A$ (反证律, 或者 $\sim$ -)
- (4) 如果 $\Sigma \vdash A \rightarrow B$;
则 $\Sigma \vdash A$;
则 $\Sigma \vdash B$ ($\rightarrow$ -)
- (5) 如果 $\Sigma, A \vdash B$;
则 $\Sigma \vdash A \rightarrow B$ ($\rightarrow$ +))
- (6) 如果 $\Sigma \vdash A \wedge B$;
则 $\Sigma \vdash A$;

则 $\Sigma \vdash B$

(7) 如果 $\Sigma \vdash A$;

$\Sigma \vdash B$;

则 $\Sigma \vdash A \wedge B$

($\wedge+$)

(8) 如果 $\Sigma, A \vdash C$;

$\Sigma, B \vdash C$;

则 $\Sigma, A \vee B \vdash C$

($\vee-$)

(9) 如果 $\Sigma \vdash A$;

则 $\Sigma \vdash A \vee B$

则 $\Sigma \vdash B \vee A$

($\vee+$)

(10) 如果 $\Sigma \vdash A \leftrightarrow B$;

$\Sigma \vdash A(B)$;

则 $\Sigma \vdash B(A)$

($\leftrightarrow-$)

(11) 如果 $\Sigma, A \vdash B$;

$\Sigma, B \vdash A$;

则 $\Sigma \vdash A \leftrightarrow B$

($\leftrightarrow+$)

附录 C： 关于形式系统 L 的一些形式定义

定义 1： 符号字母表

$p, q, r, \dots$	命题符号
$\sim, \rightarrow$	联结符号
$(,)$	标点符号

定义 2： 原子公式

命题符号即为原子公式。

定义 3： 公式

- (1) 每个原子公式都是公式。
- (2) 如果 a 和 b 是公式，则 $(\sim a)$ 和 $(a \rightarrow b)$ 也是公式。
- (3) 只有通过以上规则得到的表达式才是公式。

定义 4： 公理（公理模式）

- (L1) $(A \rightarrow (B \rightarrow A))$
- (L2) $((A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C)))$
- (L3) $((\sim A \rightarrow \sim B) \rightarrow (B \rightarrow A))$

定义 5： 形式推演规则

- (1) 如果 $\Sigma \vdash_L A \rightarrow B$;
 $\Sigma \vdash_L A$;
 则 $\Sigma \vdash_L B$ (MP)

附录 D： 关于一阶谓词逻辑的一些形式定义

关于一阶谓词逻辑，这里不考虑相等关系。

定义 1： 符号字母表

$p, q, r, \dots$	个体符号
$F, G, H, \dots$	关系符号
$f, g, h, \dots$	函数符号
$u, v, w, \dots$	自由变元符号
$x, y, z, \dots$	约束变元符号
$\forall, \exists, \dots$	量词符号
$\sim, \wedge, \vee, \rightarrow, \leftrightarrow$	联结符号
$(,)$	标点符号

定义 2： 项

- (1) 个体符号和自由变元符号是项。
- (2) 如果 $t_1, t_2, \dots, t_n$ 是项，则 $f(t_1, t_2, \dots, t_n)$ 是项，其中 f 是 n 元函数符号。
- (3) 所有项的集合是由 (1) (2) 组成的。

定义 3： 原子公式

- (1) 如果 F 是 n 元关系符号，并且 $t_1, t_2, \dots, t_n$ 是项，则 $F(t_1, t_2, \dots, t_n)$ 是原子公式。
- (2) 只有通过以上规则得到的表达式才是原子公式。

定义 4： 公式

- (1) 所有的原子公式都是公式。
- (2) 如果 a 和 b 是公式，则 $(\sim a)$, $(a \wedge b)$, $(a \vee b)$, $(a \rightarrow b)$ 和 $(a \leftrightarrow b)$ 也是公式。
- (3) 如果 $A(u)$ 是公式，并且 x 不在 $A(u)$ 中出现，则 $(\forall x)A(x)$ 和 $(\exists x)A(x)$ 也是公式。
- (4) 只有通过以上规则得到的表达式才是公式。

定义 5: 形式推演规则

- (1) $A \vdash A$ (Ref.)
- (2) 如果 $\Sigma \vdash A$;
则 $\Sigma, \Sigma' \vdash A$; (+)
- (3) 如果 $\Sigma, \sim A \vdash B$;
 $\Sigma, \sim A \vdash \sim B$;
 则 $\Sigma \vdash A$ (反证律, 或者 $\sim-$)
- (4) 如果 $\Sigma \vdash A \rightarrow B$;
 $\Sigma \vdash A$;
 则 $\Sigma \vdash B$ ($\rightarrow-$)
- (5) 如果 $\Sigma, A \vdash B$;
 则 $\Sigma \vdash A \rightarrow B$ ($\rightarrow+$)
- (6) 如果 $\Sigma \vdash A \wedge B$;
 则 $\Sigma \vdash A$;
 则 $\Sigma \vdash B$
- (7) 如果 $\Sigma \vdash A$;
 $\Sigma \vdash B$;
 则 $\Sigma \vdash A \wedge B$ ($\wedge+$)
- (8) 如果 $\Sigma, A \vdash C$;
 $\Sigma, B \vdash C$;
 则 $\Sigma, A \vee B \vdash C$ ($\vee-$)
- (9) 如果 $\Sigma \vdash A$;
 则 $\Sigma \vdash A \vee B$;
 则 $\Sigma \vdash B \vee A$ ($\vee+$)
- (10) 如果 $\Sigma \vdash A \leftrightarrow B$;
 $\Sigma \vdash A(B)$;
 则 $\Sigma \vdash B(A)$ ($\leftrightarrow-$)
- (11) 如果 $\Sigma, A \vdash B$;

$\Sigma, B \vdash A;$

则 $\Sigma \vdash A \leftrightarrow B$ ($\leftrightarrow +$)

(12) 如果 $\Sigma \vdash \forall x A(x);$

则 $\Sigma \vdash A(t)$ ($\forall -$)

(13) 如果 $\Sigma \vdash A(u)$, 并且 u 不在 Σ 中出现;

则 $\Sigma \vdash \forall x A(x)$ ($\forall +$)

(14) 如果 $\Sigma, A(u) \vdash B$, 并且 u 不在 Σ 和 B 中出现;

则 $\Sigma, \exists x A(x) \vdash B$ ($\exists -$)

(15) 如果 $\Sigma \vdash A(t);$

则 $\Sigma \vdash \exists x A(x)$ (其中的 $A(x)$ 是把 $A(t)$ 中 t 的某些出现替换为 x 得到的) ($\exists +$)

附录 E： 关于形式系统 K 的一些形式定义

形式系统 K 是一阶谓词逻辑的一种变体，这里同样不考虑相等关系。

定义 1： 符号字母表

$p, q, r, \dots$	个体符号
$F, G, H, \dots$	关系符号
$f, g, h, \dots$	函数符号
$u, v, w, \dots$	自由变元符号
$x, y, z, \dots$	约束变元符号
$\forall$	量词符号
$\sim, \rightarrow$	联结符号
$(,)$	标点符号

定义 2： 项

- (1) 个体符号和自由变元符号是项。
- (2) 如果 $t_1, t_2, \dots, t_n$ 是项，则 $f(t_1, t_2, \dots, t_n)$ 是项，其中 f 是 n 元函数符号。
- (3) 所有项的集合是由 (1) (2) 组成的。

定义 3： 原子公式

- (1) 如果 F 是 n 元关系符号，并且 $t_1, t_2, \dots, t_n$ 是项，则 $F(t_1, t_2, \dots, t_n)$ 是原子公式。
- (2) 只有通过以上规则得到的表达式才是原子公式。

定义 4： 公式

- (1) 所有的原子公式都是公式。
- (2) 如果 a 和 b 是公式，则 $(\sim a)$ 和 $(a \rightarrow b)$ 也是公式。
- (3) 如果 $A(u)$ 是公式，并且 x 不在 $A(u)$ 中出现，则 $(\forall x)A(x)$ 也是公式。
- (4) 只有通过以上规则得到的表达式才是公式。

定义 5: 公理（公理模式）

$$(K1) (A \rightarrow (B \rightarrow A))$$

$$(K2) ((A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C)))$$

$$(K3) ((\sim A \rightarrow \sim B) \rightarrow (B \rightarrow A))$$

$$(K4) ((\forall x_i)A \rightarrow A), \quad x_i \text{ 不在 } A \text{ 中自由出现}$$

$$(K5) ((\forall x_i)A(x_i) \rightarrow A(t)), \quad \text{其中 } t \text{ 是项, 它对 } A(x_i) \text{ 中的 } x_i \text{ 是自由的}$$

$$(K6) (\forall x_i)(A \rightarrow B) \rightarrow (A \rightarrow (\forall x_i)B), \quad \text{其中 } A \text{ 不包含 } x_i \text{ 的自由出现}$$

定义 6: 形式推演规则

$$(1) \text{ 如果 } \Sigma \vdash_K A \rightarrow B;$$

$$\Sigma \vdash_K A;$$

$$\text{则 } \Sigma \vdash_K B$$

(MP)

$$(2) \text{ 如果 } \Sigma \vdash_K A;$$

$$\text{则 } \Sigma \vdash_K (\forall x_i)A$$

(全称概括规则)

附录 F： 关于 π 演算的一些形式定义

这里使用 BNF 表示法。

定义 1： 名

名： x, y, z

定义 2： 动作项

动作项： $:= \bar{x} z.P$ （沿着 x 发送 z ）

$x y.P$ （沿着 x 接收任何 y ）

定义 3： π 项

π 项： $:= A_1 + A_2 + \dots + A_n$ （交互动作）

$P_1 | P_2$ （并发）

$P_1 \parallel P_2$ （并行）

$\nu y.P$ （限制）

$!P$ （复制）

定义 4： 计算的基本规则

$x y.P_1[y] | z.P_2 \rightarrow P_1[Z] | P_2$

附录 G： χ 演算与 π 演算的比较

χ 演算可以堪称是对 π 演算的简化，主要有如下一些不同（其实是关于 χ 演算的很多内容都只能参考一些英文文献，这些英文文献也不好找，所以这里暂且不给出 χ 演算的一些形式定义）：

（1） χ 演算整合了 π 演算的通道的输入端和输出端（即用 $[x] y.P$ 整合了 $\bar{x} z.P$ 和 $x y.P$ ，其中 $[x]$ 可取 x 或 $\bar{x}$ ）。

（2）在 π 演算中，存在有两类局部名， χ 演算中只有一类，去除了 π 演算中的前类局部名，输入和输出是对称的。

（3） χ 演算对通讯机制的刻画更加直观、简洁和深刻，不需要边侧条件。

（4） χ 演算对互模拟概念的定义更容易理解。